

Claude Code Telemetry Analytics

An end-to-end Streamlit and DuckDB platform that turns nested JSONL telemetry into reproducible product, engineering, cost, tool, reliability, and environment insights.

Technical assessment presentation

Introduction To The App

Purpose, value, and users of the telemetry analytics platform

02

- Turns raw Claude Code telemetry into business-readable usage, cost, reliability, and adoption insights.
- Helps product leaders understand adoption patterns, model mix, and engagement depth.
- Helps engineering and operations teams monitor API errors, tool success, environment versions, and latency.
- Runs locally with generated data, no secrets, and one primary demo command.

Why companies would use it



Adoption

Which teams and roles use AI coding tools?



Cost

Where are spend and token volume concentrated?



Reliability

Are API and tool failures creating operational risk?



Governance

Can telemetry be traced back to raw records?

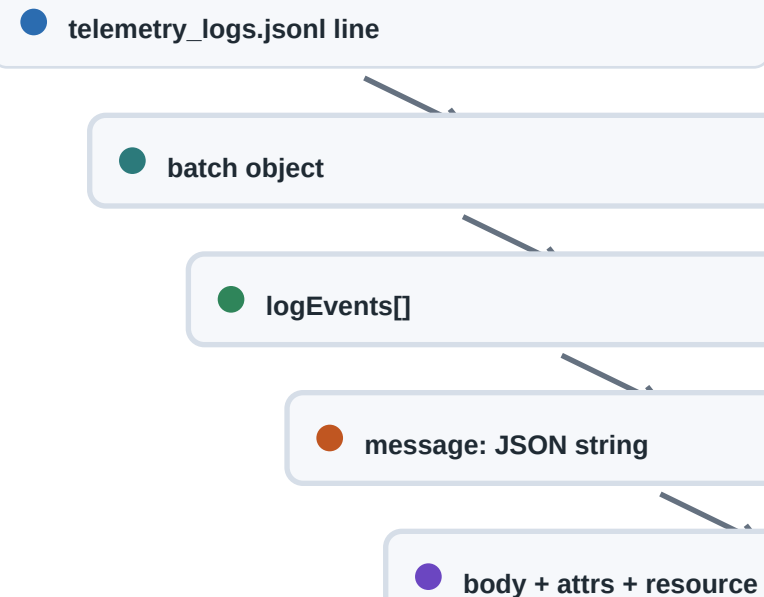
Problem And Dataset

03

The core challenge is nested telemetry, not just dashboard rendering

- The source telemetry is semi-structured and nested, so parsing correctness is central to metric trust.
- Each JSONL line is a CloudWatch-style batch, not a single event.
- Every `logEvents[].message` field is itself a JSON-encoded telemetry event.
- Employee enrichment is joined by email to support practice, level, and location analysis.

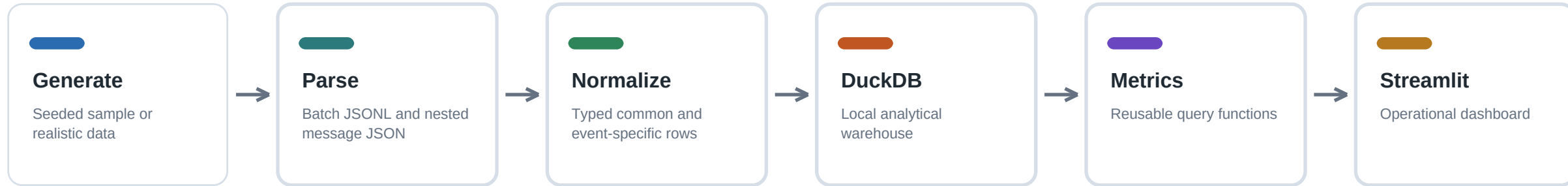
Nested JSONL contract



Architecture

04

A layered local analytics stack: generator, parser, warehouse, metrics, dashboard



Normalized model

raw_log_batches

raw_log_events

events

api_requests

api_errors

tool_decisions

tool_results

user_prompts

employees



AGENTS.md

Defines the assignment goal, nested JSONL contract, normalized model, expected metrics, commands, and tests.



telemetry-analysis skill

Keeps domain-specific parsing, normalization, dashboard, metric, validation, and documentation rules in the repo.



Development effect

Reduced drift between docs, code, tests, dashboard labels, and metric denominator definitions.

Validation is non-fatal and raw records stay traceable

Two-stage parser

Parse JSONL batches, iterate logEvents, then parse each message field as nested JSON.

Typed boundary

Convert timestamps, costs, tokens, durations, attempts, byte sizes, and booleans in normalization.

Traceability

Retain raw batch, raw log event, raw message, attributes, and resource payloads for debugging.

Employee enrichment

Join employees.csv by user_email to enable practice, level, and location analysis.

Validation coverage

- ✓ Missing input files
- ✓ Malformed JSONL lines
- ✓ Invalid logEvents
- ✓ Invalid nested message JSON
- ✓ Missing common fields
- ✓ Numeric/timestamp coercion failures
- ✓ Employee mismatches

Dashboard Walkthrough

07

Actual app navigation and what each dashboard section lets the user do

Dashboard navigation

The user can move between six operational sections from the top tab bar. Each section uses the same global filter context and metric/query layer.

Overview Model Usage Team/User Insights Tool Behavior Reliability Environment

Overview

Executive summary: active users, sessions, prompts, requests, cost, token totals, adoption trends, and reliability KPIs.

Model Usage

Compare models by spend, request count, latency, token mix, and model-level API error rate.

Team/User Insights

Break usage down by practice, level, and location to identify adoption patterns across cohorts.

Tool Behavior

Review tool decisions, acceptance rate, success rate, duration, and output-size patterns.

Reliability

Inspect API errors, error-rate denominators, status-code mix, retry attempts, and model breakdowns.

Environment

Analyze terminal, OS, service version, and host architecture distribution for operational support.

Key Findings From Generated Sample Data

08

Sample-data findings are produced by implemented metrics, not manual spreadsheet math



ACTIVE USERS

5



SESSIONS

20



EVENTS

1,558



API REQUESTS

407



COST

\$17.11



TOKENS

20.8M



API ERROR RATE

1.23%



TOOL SUCCESS

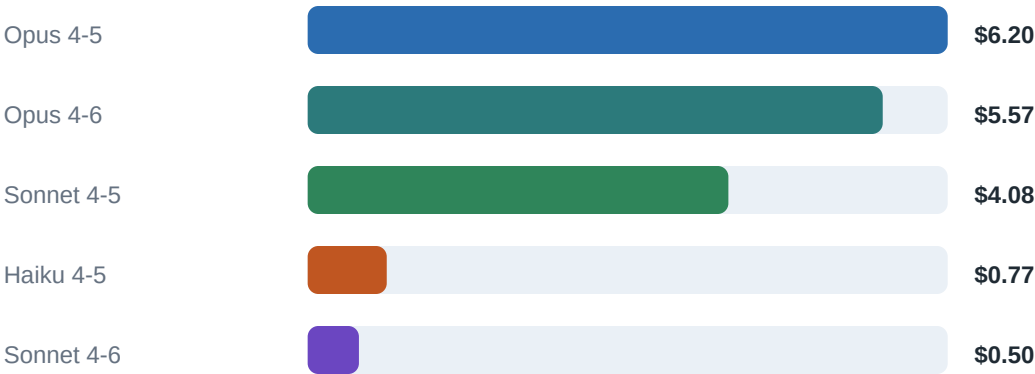
97.8%

Interpretation: the generated sample validates the pipeline and dashboard. It shows meaningful adoption, high tool success, low API error volume, and token volume dominated by cache-read usage.

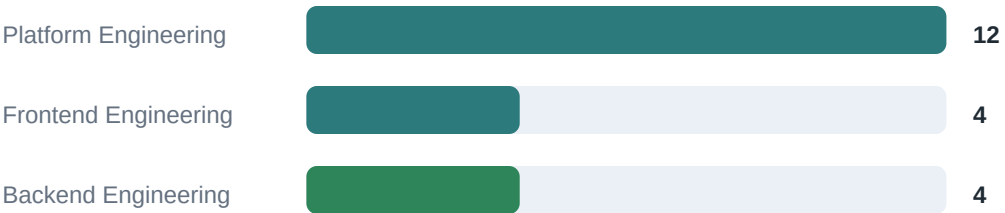
Model And Team Insights

Cost, model mix, and cohort usage help leaders decide where to focus

Model cost by generated spend



Sessions by practice

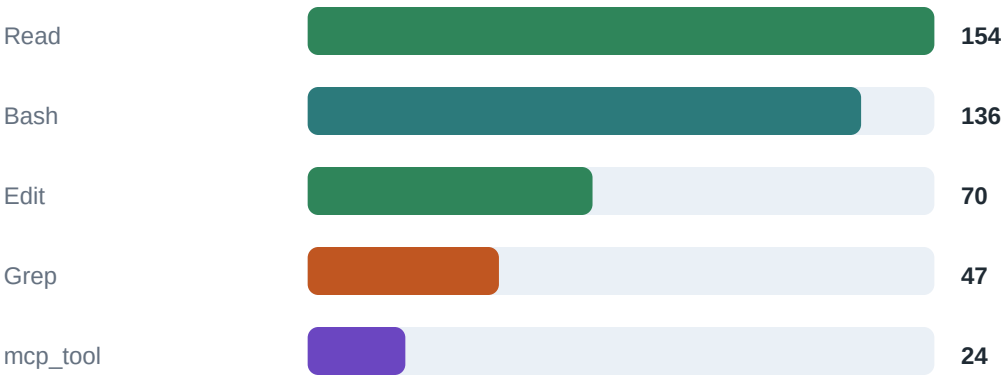


Example readout: Opus 4-5 leads generated spend at \$6.20; Platform Engineering leads sessions with 12.

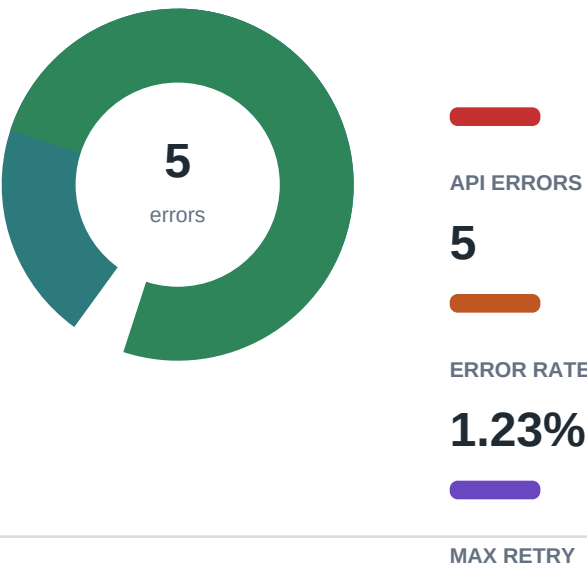
Tool Behavior And Reliability

Tool adoption and failure patterns help operations teams find friction

Top tools by decision count



API error status mix



Example readout: Read is the most common tool with 154 decisions and 99.3% result success.



DuckDB local warehouse

Fast, simple, and easy to reproduce, but not a multi-user production warehouse.



Synthetic dataset

Excellent for deterministic validation, but generated findings are not production business conclusions.



Streamlit UI

Efficient for dashboard delivery, but not a governed enterprise BI deployment.



Prompt redaction

Protects sensitive content; analysis focuses on prompt counts and lengths rather than prompt text.

1

Incremental ingestion

Track source files, batch IDs, and partition refreshes instead of full DB rebuilds.

2

Session intelligence

Add duration distributions, high-cost outliers, depth metrics, and anomaly detection.

3

Productionization

Package with Docker Compose, CI tests, linting, skill validation, and deployment docs.

4

Stakeholder reporting

Add exportable dashboard snapshots and scheduled reports for leadership reviews.

What the project demonstrates

- Correct nested JSONL parsing and typed normalization with raw-record traceability.
- A local DuckDB warehouse aligned to a normalized telemetry model.
- Reusable metric functions that keep dashboard calculations testable.
- A Streamlit dashboard organized around adoption, cost, model usage, team insights, tools, reliability, and environment.
- A deterministic sample run with 1,558 events, 407 API requests, and zero ingestion validation errors.

Ready to demo with make demo